

1. What is Dependency Injection (DI)?

Dependency Injection is a design pattern where an object receives its dependencies (other objects it needs to work) from an external source rather than creating them itself.

- Instead of writing `new UserService()` inside your controller, you inject it (via constructor, setter, or field).
- This decouples classes, making them loosely coupled, easier to test, and more maintainable.

Example (without DI):

java

```
class Car {  
    private Engine engine = new PetrolEngine(); // tightly coupled  
}
```

Example (with DI):

java

```
class Car {  
    private Engine engine;  
  
    public Car(Engine engine) { // injected from outside  
        this.engine = engine;  
    }  
}
```

2. What is a Bean?

A Bean is simply a Java object that is managed by the Spring IoC container.

- Spring creates, wires, and destroys these objects for you.
- You define beans using:
 - Annotations: `@Component`, `@Service`, `@Repository`, `@Controller`
 - XML configuration (`<bean>`)
 - Java config (`@Bean` inside `@Configuration` class)

Example:

java

```
@Component // tells Spring to create and manage this as a bean
public class UserService {
    // ...
}
```

💡 In short: Any plain Java object that Spring instantiates and manages is a Bean.

3. What is an IoC Container and its Types?

IoC (Inversion of Control) Container is the core engine of Spring that:

- Creates and manages Beans (objects)
- Performs Dependency Injection (wires beans together)
- Manages the lifecycle (creation → usage → destruction) of beans

Think of it as a "runtime factory" that holds all your beans and injects them where needed.

Types of IoC Containers in Spring:

Container	Description	When to use
1. BeanFactory (Legacy)	The basic IoC container. Lazily loads beans (creates them only when requested). Provides only fundamen	Rarely used today; mostly in lightweight or resource-constrained environments.

	tal DI features.	
2. ApplicationContext (Modern)	The advanced IoC container. Eagerly loads beans at startup. Extends BeanFactory with enterprise features like: • Internationalization (i18n) • Event publishing • AOP support • Web-aware context (WebApplicationContext)	This is the default and recommended container for all real-world Spring/Spring Boot projects.

Common implementations of ApplicationContext:

- `ClassPathXmlApplicationContext` – loads XML config from classpath
 - `FileSystemXmlApplicationContext` – loads XML from file system
 - `AnnotationConfigApplicationContext` – loads Java-based config (most common in modern Spring)
 - `WebApplicationContext` – used in web applications (Spring MVC)
-

How They Connect (The Big Picture):

1. IoC Container (`ApplicationContext`) holds all your Beans.
 2. When one Bean needs another, the container uses Dependency Injection to provide it.
 3. You just annotate classes and let the container do the wiring — that's Spring's magic.
-

Real-world analogy:

- IoC Container = a restaurant kitchen manager
- Beans = ingredients and chefs
- Dependency Injection = the manager assigns the right chef and ingredients to each order automatically — you don't call chefs directly; the manager does it for you.